# CartPole-v1

Policy gradient with and without a baseline

The primary basic-policy-gradient run achieved **500.00 ± 0.00** over 500 evaluation episodes. Adding a learned baseline reduced measured gradient variance at a fixed policy by **84.22%**, but produced lower final evaluation scores in this training experiment. All three training seeds are reported below.

## 1 Environment, notation, and objective

The environment is `CartPole-v1`, run through Gymnasium 1.2.2, the maintained Gym successor. It uses the default reward of +1 per decision, including the terminal transition, and a 500-step limit. Actions push the cart left or right. The four observations are cart position, cart velocity, pole angle, and pole angular velocity. Physical failure occurs when the cart or pole crosses the allowed boundary.[1]

**Table 1:** Notation used in the algorithm.

| Symbol | Meaning |
|---|---|
| $s_t$ | Four-component observation before decision $t$. |
| $a_t$ | Sampled action, 0 (left) or 1 (right). |
| $r_t$ | Reward received after action $a_t$; normally +1. |
| $T$ | Number of decisions in the current episode, at most 500. |
| $\theta$ | All weights and biases of the policy neural network. |
| $\pi_\theta(a \mid s)$ | Probability of selecting action $a$ in state $s$. |
| $\gamma$ | Discount factor, fixed at 0.95. |
| $G_t$ | Discounted reward still to be collected from decision $t$. |

The discounted episodic objective and reward-to-go are

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right], \qquad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k. \tag{1}$$

Here $\mathbb{E}_{\pi_\theta}$ means an average over episodes generated by the policy and environment. A larger $J$ is better. Compute $G_t$ backward: start with zero after the last reward and repeatedly apply $G_t = r_t + \gamma G_{t+1}$.

**Example.** Three rewards of 1 give reward-to-go values $(2.8525, 1.95, 1)$ when $\gamma = 0.95$. These are the learning targets. The *plotted episode score* is instead the undiscounted sum $\sum_t r_t$, which is 3 in this example and at most 500 in CartPole.

All experiments used **CPU only and zero GPUs** on deepdish5. The code explicitly hides CUDA devices and uses one PyTorch/BLAS CPU thread.

[1] Environment specification: `https://gymnasium.farama.org/environments/classic_control/cart_pole/`.

## 2 Implementation and comparison protocol

### 2.1 Basic REINFORCE

The policy has four inputs, one hidden layer of 64 tanh units, and two output logits (unnormalized action scores). A categorical distribution converts the logits into action probabilities. Actions are sampled during both training and evaluation.

After collecting $B = 32$ complete episodes, minimize the sampled loss

$$\mathcal{L}_{\text{actor}} = -\frac{1-\gamma}{B} \sum_{i=1}^{B} \sum_{t=0}^{T_i-1} \gamma^t \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)})(G_t^{(i)} - b_t^{(i)}). \tag{2}$$

The index $i$ identifies an episode, and $T_i$ its length. In the basic method, $b_t^{(i)} = 0$. The gradient increases the probability of actions associated with larger discounted returns. Reward-to-go removes rewards occurring before an action, which cannot be affected by that action. The outer $\gamma^t$ is retained for the stated discounted episodic objective.

Adam uses the specified learning rate 0.01. The fixed factor $1 - \gamma = 0.05$ rescales the objective without changing its optimum. The gradient-norm cap is 10; it was never reached in any recorded run. Returns and advantages are *not* normalized, so no implicit baseline is introduced in Part 1.

### 2.2 Learned baseline for Part 2

The baseline $b_\psi(s_t, t)$ predicts $G_t$, using a separate neural network with parameters $\psi$. It receives the four state components and elapsed time divided by 500, followed by 64 tanh units and one scalar output. Time is included because the remaining episode horizon can affect expected return.

Before each actor update, baseline predictions are computed and detached: the actor treats them as fixed numbers, and its loss does not update $\psi$. Afterward, the baseline receives five Adam updates at learning rate 0.01 to minimize the mean squared difference between predictions and full Monte Carlo targets $G_t$. There is no temporal-difference bootstrapping.

An action-independent baseline leaves the expected policy gradient unchanged because, conditional on the state and history,

$$\sum_a \pi_\theta(a \mid s) b \nabla_\theta \log \pi_\theta(a \mid s) = b \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0. \tag{3}$$

The symbol $\nabla_\theta$ denotes derivatives with respect to the policy parameters. Baseline fitting uses information from earlier batches before the current actions are collected, avoiding dependence of the current baseline on its sampled action.

### 2.3 Training and evaluation protocol

Each method trains for 8,000 episodes (250 updates) using seeds 1668, 1669 and 1670. Seed 1668 was fixed as the primary run before full training. A 1,024-episode basic-method pilot checked functionality at learning rate 0.01 and is archived separately. The full runs restart from their seeds. Actor initialization, reset-seed schedule, learning rate and episode budget are matched across methods.

Every final checkpoint is evaluated over 500 fresh episodes, with reset seeds 9000000–9000499 shared across methods. Training does not use these resets. No checkpoint is selected using evaluation performance. Evaluation keeps stochastic actions and never updates weights. Standard deviations reported below use the population formula over the 500 episode scores.

## 3 Part 1(a): basic policy gradient results

For the primary seed, the final policy achieved 500 in every evaluation episode: mean **500.000**, standard deviation **0.000**. All 500 episodes reached the time limit. This is strong evidence for that particular model on the evaluated reset distribution, not proof that every training seed converges.

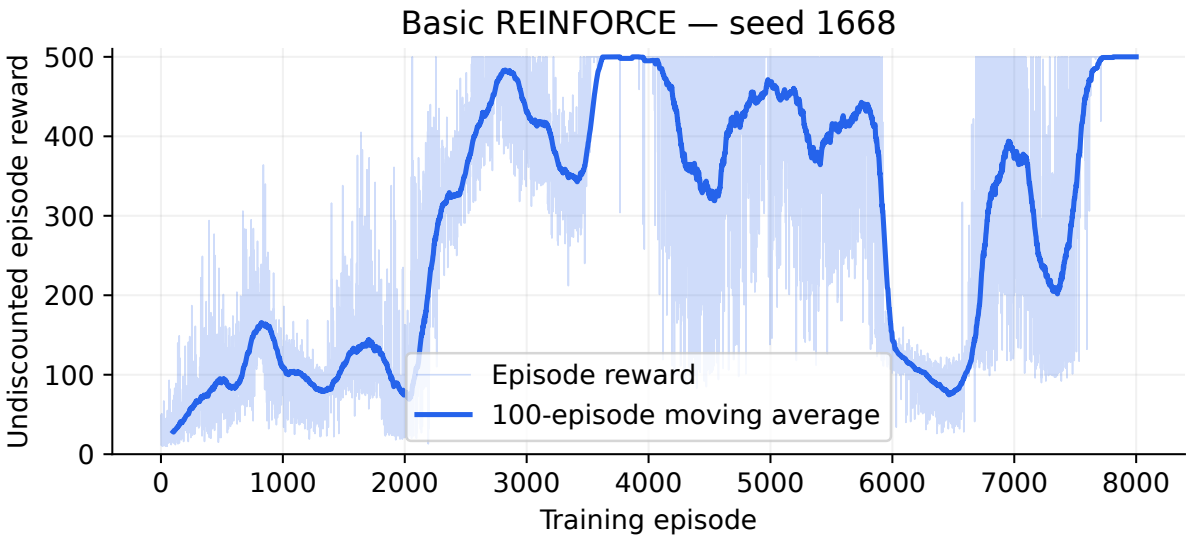



**Figure 1:** Basic REINFORCE: raw episode rewards and the trailing 100-episode moving average, starting at episode 100. Training seed 1668.

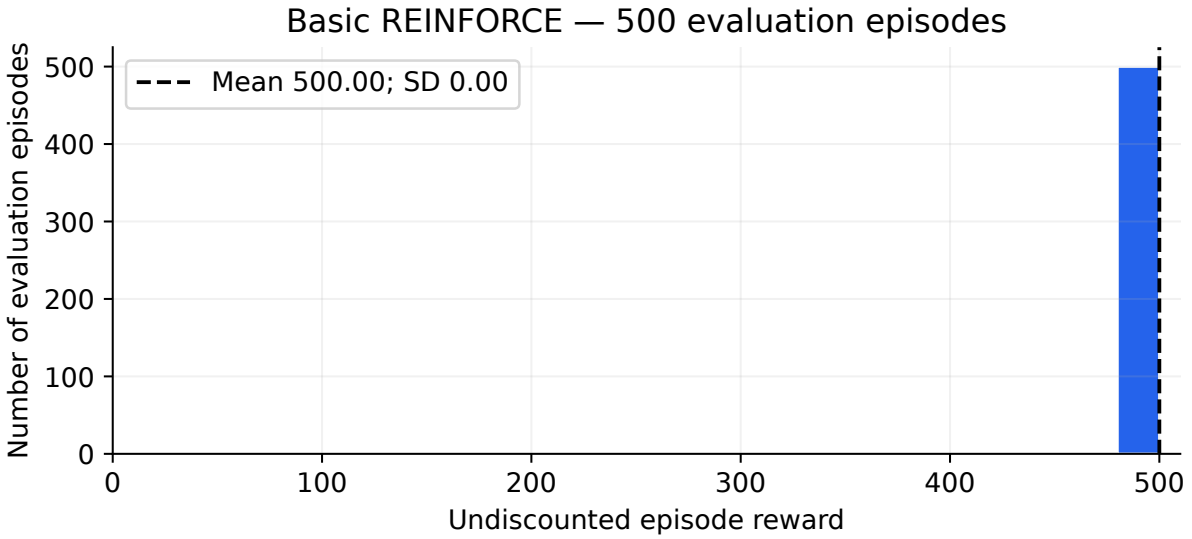



**Figure 2:** Basic REINFORCE: histogram of 500 fresh evaluation scores. The mass lies in the final bin because all scores are 500.

A uniform random policy scored $22.782 \pm 12.058$ over 500 episodes. The untrained primary neural policy scored $20.926 \pm 11.467$. These references support a substantial learned improvement rather than good performance from initialization alone.

## 4 Part 2: results with a learned baseline

For the primary seed, the baseline model scored **260.308 ± 57.013**. Four of its 500 evaluation episodes reached the time limit; 496 ended through physical failure. It improved substantially over the random and untrained references but underperformed the primary basic-method model.

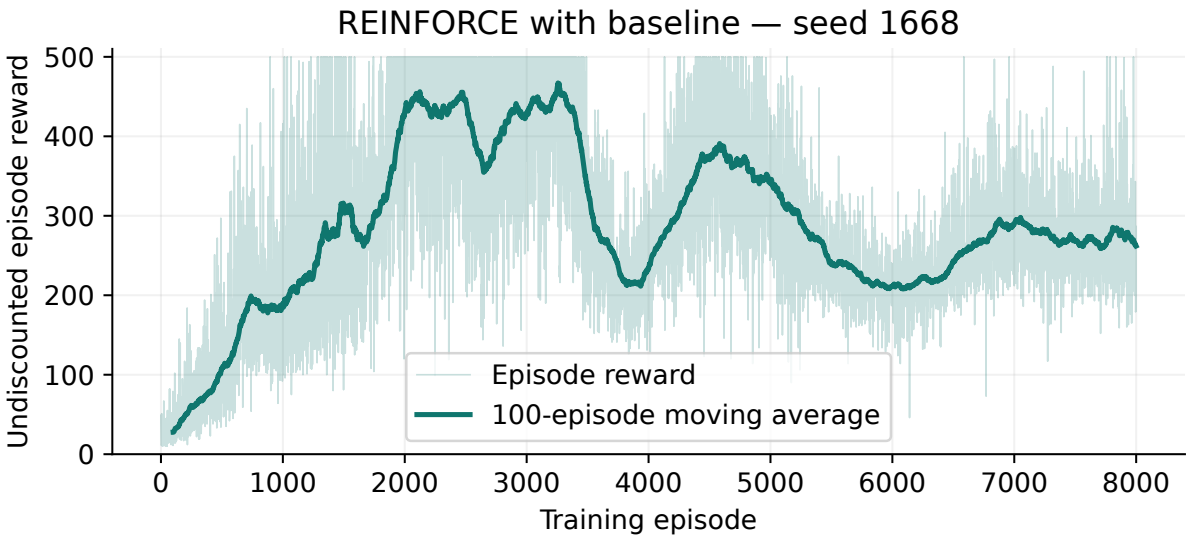



**Figure 3:** REINFORCE with learned baseline: episode rewards and the trailing 100-episode moving average. Training seed 1668.

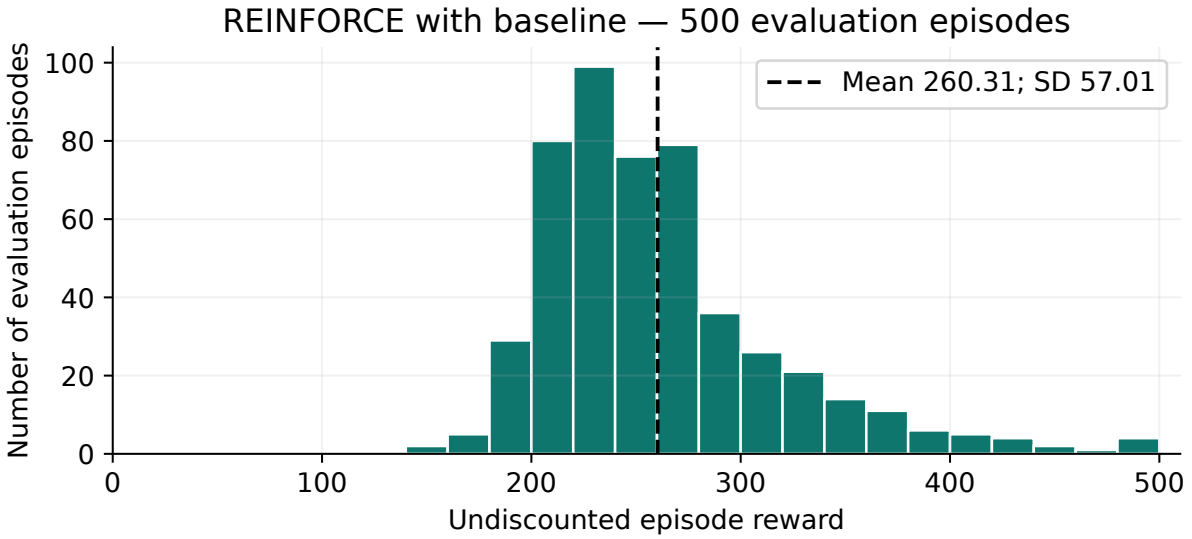



**Figure 4:** Learned-baseline policy: histogram of 500 fresh evaluation scores, with mean 260.308 and population standard deviation 57.013.

The baseline method receives exactly the same number of environment episodes as the basic method, but performs additional computation to fit its value predictor. Lower estimator variance alone does not guarantee a better final policy under a fixed training budget.

## 5 Across-seed results and the variance experiment

**Table 2:** Every final model, each evaluated on 500 fresh episodes. Within-row SD describes variation among episodes for one fixed model.

| Method | Training seed | Mean score | SD | Fraction at 500 |
|---|---|---|---|---|
| No baseline | 1668 | 500.000 | 0.000 | 1.000 |
| No baseline | 1669 | 271.824 | 28.856 | 0.000 |
| No baseline | 1670 | 483.886 | 65.298 | 0.928 |
| Learned baseline | 1668 | 260.308 | 57.013 | 0.008 |
| Learned baseline | 1669 | 360.516 | 79.141 | 0.100 |
| Learned baseline | 1670 | 349.940 | 76.321 | 0.092 |

The mean of the three seed-specific evaluation means was 418.570 without the baseline and 323.588 with it. The corresponding sample SDs across seed means were 127.341 and 55.057. These are different quantities from within-model episode SD. Three seeds show sensitivity to initialization and sampling but are insufficient for a broad statistical claim. The baseline was better for seed 1669, but worse for the other two seeds and lower on average.

### 5.1 Does the baseline actually reduce gradient variance?

Comparing training reward variance cannot isolate this question because the policies change. Instead, freeze one initial policy. Fit a baseline on 512 independent episodes, then calculate both gradient estimators on the *same* 512 different trajectories. For each policy parameter, calculate its sample gradient variance, and sum these variances. This sum is the trace of the gradient covariance matrix.

**Table 3:** Paired gradient-variance measurement at one frozen policy.

| Estimator | Sum of parameter variances |
|---|---|
| No baseline | 5.338262 |
| Learned baseline | 0.842421 |

The measured reduction was $1 - 0.842421/5.338262 = 84.22\%$. The distance between the two empirical mean gradients was 0.0474, compared with a root-sum-squared standard error of 0.1176 for their paired difference. Finite-sample mean agreement is consistent with the analytic baseline-invariance property; it is not itself a proof. The experiment demonstrates variance reduction at this frozen initial policy, not at every later training policy.

### 5.2 Discounted objective versus episode score

With reward +1 each step, a length-$T$ episode has discounted return

$$\frac{1 - 0.95^T}{1 - 0.95}, \tag{4}$$

which approaches 20. The primary baseline policy's average discounted evaluation return was 19.999743, while the basic model's was approximately 20.000000, despite their large difference in undiscounted scores. Discounting therefore makes extra survival late in an episode a weak objective signal. This observation explains a limitation of the required objective, without establishing the cause of the baseline method's lower final score.

## 6 Edge cases and correctness validation

### 6.1 Predictable calculations and policy updates

**Discounting.** For rewards $(1, 1, 1)$, backward calculation produces $(2.8525, 1.95, 1)$; implementation values match to floating-point precision. Setting $\gamma = 0$ produces the immediate rewards exactly.

**Exact policy-gradient example.** In a two-step toy environment, let both actions be sampled from a Bernoulli probability $p = \text{sigmoid}(\theta)$ and let each reward equal its binary action. The exact objective is $(1 + \gamma)p$, with derivative $(1 + \gamma)p(1 - p)$. Enumerating all four possible action sequences gave gradient 0.471188756 with or without a constant baseline; the analytic derivative was 0.471188748. This checks the return-to-go estimator, outer discount factor, and baseline invariance.

**Gradient direction.** A positive-reward update for one selected action increased its probability from 0.500000 to 0.502500. A wrong sign in the actor loss would reverse this direction.

### 6.2 Environment end conditions and reproducibility

Fixed reset seeds and a fixed ten-action sequence reproduced the same states exactly. Deliberately placing the cart beyond the position boundary and the pole beyond its angle boundary separately produced physical termination. Each terminating step still delivered reward +1.

A one-step time-limit experiment produced truncation without physical termination. Training and evaluation stop on either flag. Because the objective is a complete finite episode, returns end at that boundary and do not bootstrap from a later state. The environment is reset before another episode, and return accumulation never crosses episode boundaries.

### 6.3 Inspecting ten actual decisions

The file `validation/first_ten_steps.csv` records the four state components, probability of moving right, sampled action, reward and end flags. The primary basic-policy-gradient checkpoint was used, with reset seed 9100000 and action-sampling seed 123456. The first decisions illustrate the distinction between action probability and action selection:

**Table 4:** Excerpt of the saved trace. Angles are in radians.

| $t$ | Position | Velocity | Angle | Angular velocity | $P$(right) | Action |
|---|---|---|---|---|---|---|
| 0 | 0.01276 | -0.01188 | -0.04659 | -0.03077 | 0.1019 | Left |
| 1 | 0.01252 | -0.20630 | -0.04721 | 0.24686 | 0.8335 | Right |
| 2 | 0.00840 | -0.01054 | -0.04227 | -0.06033 | 0.0700 | Left |
| 3 | 0.00818 | -0.20503 | -0.04348 | 0.21872 | 0.7711 | Left |

All four rewards were 1. At decision 3, the less likely left action was sampled even though the probability of right was 0.7711. This is valid stochastic behavior, not a mismatch between the network and environment. The full trace permits further manual inspection rather than relying only on an aggregate learning curve.

## 7 Reproduction, limitations, and additional experiments

The training code, final and initial model checkpoints, configurations, evaluation rewards and logs are saved in `mlds_assignment1_cartpole`. On deepdish5, the environment is already installed. Run:

```
cd ~/mlds_assignment1_cartpole
source ~/mlds_assignment1_cartpole_env/bin/activate
python cartpole.py suite --episodes 8000 --lr 0.01
python validate.py
```

This regenerates and overwrites the six run folders; preserve the original results if desired. To inspect saved results without training, open `cartpole_plots.ipynb` and run all cells. The notebook needs only NumPy, Matplotlib and Jupyter. It generates the required reward curves, 100-episode moving averages and 500-episode histograms, as well as additional comparison figures.

Per-run `config.json` files record the actual software and settings: Python 3.12, PyTorch 2.14.0+cu130, NumPy 2.5.3 and Gymnasium 1.2.2. Although the installed PyTorch supports CUDA, all experiments used CPU. Full validation evidence is in `validation/validation.json`, with paired sample gradients in `validation/gradient_variance.npz`.

### 7.1 Limits of the evidence

The primary basic-policy-gradient model reached the maximum in all 500 trials, but this did not occur for every seed. Likewise, the baseline's variance reduction at a frozen initial policy did not imply better final performance. The learned value approximation, finite data and evolving policy all merit further investigation. The experiments do not establish which factor caused the difference, and no evaluation-based hyperparameter or checkpoint selection was used to improve the reported results.

### 7.2 With an additional three months

First, run more training seeds and tune learning rate, episode batch size and critic-fitting frequency using *separate validation seeds*. Reserve another reset-seed set for final evaluation. This would distinguish reproducible improvements from favorable individual runs.

Second, repeat paired variance measurements at early, middle and late policy checkpoints. Compare a constant baseline, the current state/time baseline, and different critic capacities. Measure value-prediction error alongside gradient variance and average gradient direction. These experiments would test whether inaccurate or outdated predictions limit the baseline during training.

Finally, study robustness under perturbed reset states, pole lengths and cart/pole masses. Evaluate stochastic and greedy policies separately, and measure the impact of truncation and the required discount factor. Any alternative discounting should be an additional sensitivity experiment, with the assignment's $\gamma = 0.95$ results retained as the main comparison.